

SuperPlots em Python (Matplotlib + Seaborn)

Curso de HCI/HCA

NanoCell Interactions Lab

2026-07-21

Índice

0.1	Etapa 1: Criar o ambiente Pixi	2
0.2	Etapa 2: Instalar o ambiente	3
0.3	Etapa 3: Abrir o projeto no programa de sua preferência	4
0.4	Opcional: abrir um shell com o ambiente ativado	4
0.5	Verificar a instalação	4
0.6	Etapa 4: Organizar os dados	5
0.7	Etapa 5: Carregar e verificar os dados	6
0.8	Etapa 6: Calcular as médias das replicatas	6
0.9	Etapa 7: Escolher e executar o teste estatístico adequado	6
0.10	Etapa 8: Criar o SuperPlot	8
0.11	Etapa 9: Salvar figuras prontas para publicação	10
0.12	Etapa 10: Transformar o código em uma função reutilizável	10
0.13	Etapa 11: Variações comuns	13
0.14	Etapa 12: Recomendações de reprodutibilidade	13

Este tutorial mostra como criar **SuperPlots**, figuras que comunicam simultaneamente a variabilidade entre células e a reprodutibilidade entre experimentos.

Ideia central: represente todas as medidas em nível celular como pontos pequenos, sobreponha as médias das replicatas como marcadores maiores, contornados e coloridos por replicata, e realize a análise estatística sobre as médias das replicatas, não sobre as células individuais.

Para ilustrar a importância que a visualização transparente dos dados adquiriu nos padrões atuais de publicação, considere o seguinte comentário que recebemos após submeter um manuscrito ao *Journal of Cell Science*:

Como boa prática e para aumentar a transparência, permitindo uma avaliação mais adequada da qualidade dos dados e de quanto eles sustentam as conclusões, recomendamos fortemente que você:

1. Utilize gráficos que permitam ao leitor visualizar a distribuição real dos dados, como boxplots, gráficos de caixa e bigodes e SuperPlots. Consulte [PMID: 32346721](#) para mais informações. Ao utilizar gráficos de caixa e bigodes, informe também o que os bigodes representam. Consulte [here](#) para orientações adicionais.
2. Utilize métodos estatísticos apropriados, considerando como tamanho amostral as replicatas biológicas, e não as replicatas técnicas.
3. Caso utilize western blots, forneça uma figura suplementar adicional, em um único arquivo PDF ou TIFF. Essa figura não será contabilizada no limite de figuras suplementares. Caso ela tenha sido enviada na submissão com o nome *Blot transparency*, numere-a e combine-a com o restante do material suplementar em PDF. Cite essa figura em Materiais e Métodos, preferencialmente no parágrafo referente aos western blots.

0.1 Etapa 1: Criar o ambiente Pixi

Este tutorial utiliza o [Pixi](#) para criar um ambiente Python reproduzível. O Pixi instala o Python e todos os pacotes necessários a partir de um único arquivo de configuração, portanto não é necessário criar ou ativar manualmente um ambiente Conda. Caso esteja utilizando o ambiente Pixi fornecido para o curso de HCI/HCA, você não precisa seguir as etapas de criação do ambiente abaixo, pois ele já contém todos os pacotes exigidos neste tutorial. Caso esteja trabalhando fora do ambiente do curso ou testando o tutorial de forma independente, siga as instruções a seguir.

Crie uma nova pasta para o tutorial e entre nela:

```
mkdir superplots-tutorial
cd superplots-tutorial
```

Crie um arquivo chamado `pixi.toml` com o seguinte conteúdo:

```
[workspace]
name = "superplots-tutorial"
version = "0.1.0"
description = "Minimal environment for plotting complete datasets and SuperPlots"
channels = ["conda-forge"]
platforms = ["linux-64", "osx-64", "osx-arm64", "win-64"]

[dependencies]
python = "≥3.10,<3.13"
pandas = "≥2.0"
numpy = "≥1.24"
matplotlib = "≥3.7"
seaborn = "≥0.12"
scipy = "≥1.11"
ipykernel = "≥6.0"
jupyterlab = "≥4.0"

[tasks]
notebook = "jupyter lab"
```

A pasta deverá conter:

```
superplots-tutorial/
└─ pixi.toml
```

0.2 Etapa 2: Instalar o ambiente

Na pasta que contém o arquivo `pixi.toml`, execute:

```
pixi install
```

O Pixi criará o ambiente e gerará um arquivo `pixi.lock` contendo as versões resolvidas dos pacotes. Mantenha os arquivos `pixi.toml` e `pixi.lock` junto ao tutorial para preservar a reprodutibilidade do ambiente.

0.3 Etapa 3: Abrir o projeto no programa de sua preferência

0.3.1 Etapa 3a: Abrir o projeto no VSCode

Abra a pasta do tutorial no VSCode:

```
code .
```

0.3.2 Etapa 3b: Abrir o projeto no JupyterLab

Execute:

```
pixi run notebook
```

Esse comando inicia o JupyterLab dentro do ambiente Pixi, com todos os pacotes necessários disponíveis.

Como alternativa, execute o JupyterLab diretamente:

```
pixi run jupyter lab
```

0.4 Opcional: abrir um shell com o ambiente ativado

Não é necessário ativar o ambiente antes de executar o tutorial. O Pixi pode executar comandos diretamente com `pixi run`.

Caso prefira trabalhar em um shell com o ambiente ativado, use:

```
pixi shell
```

Para sair do ambiente, execute:

```
exit
```

0.5 Verificar a instalação

Crie um notebook e execute:

```
import matplotlib
import numpy
import pandas
import scipy
import seaborn

print("Ambiente pronto.")
```

Caso a célula seja executada sem erros, o ambiente está pronto para o tutorial.

0.6 Etapa 4: Organizar os dados

A tabela deve estar em formato longo e conter, no mínimo, as seguintes colunas:

- Treatment – identificação da condição, por exemplo, Control ou Drug;
- Replicate – identificação da replicata biológica, por exemplo, R1, R2 ou R3;
- Value – medida obtida em nível celular, por exemplo, Speed ou Area.

Caso a coluna de medida tenha outro nome, como Speed, ajuste o código abaixo.

0.6.1 Modelo mínimo de CSV para copiar em `combined.csv`

```
Treatment,Replicate,Value
Control,R1,10.1
Control,R1,11.3
Control,R1,9.9
Control,R2,12.0
Control,R2,11.6
Drug,R1,7.2
Drug,R1,8.1
Drug,R2,9.0
Drug,R2,8.6
```

Dica: Em desenhos pareados, como Controle e Tratamento avaliados nas mesmas linhagens ou nos mesmos dias experimentais, certifique-se de que os mesmos identificadores de Replicate estejam presentes em cada tratamento.

0.7 Etapa 5: Carregar e verificar os dados

```
import pandas as pd

combined = pd.read_csv("combined.csv")
assert {"Treatment", "Replicate", "Value"}.issubset(combined.columns), "Columns missing!"

print(combined.groupby(["Treatment", "Replicate"]).size())
```

- Verifique se há erros de digitação nos nomes dos tratamentos ou das replicatas.
- Confirme que as replicatas que devem ser pareadas têm os mesmos identificadores em todos os tratamentos.

0.8 Etapa 6: Calcular as médias das replicatas

```
ReplicateAverages = (
    combined
    .groupby(["Treatment", "Replicate"], as_index=False)["Value"]
    .mean()
    .rename(columns={"Value": "RepMean"})
)

ReplicateAverages.head()
```

0.9 Etapa 7: Escolher e executar o teste estatístico adequado

Os SuperPlots enfatizam que a análise estatística deve ser realizada sobre as médias das replicatas.

- Teste t pareado: as mesmas replicatas estão presentes nos dois tratamentos, por exemplo, R1, R2 e R3 em Controle e Tratamento.
- Teste t de Welch (não pareado): os identificadores das replicatas são diferentes ou os grupos têm tamanhos amostrais distintos.

```
import numpy as np
from scipy import stats

# Exemplo: comparar exatamente dois tratamentos, como "Control" e "Drug"
treatments = ["Control", "Drug"] # altere conforme necessário

pivot = ReplicateAverages.pivot_table(
    index="Replicate",
    columns="Treatment",
    values="RepMean"
)

# Verificar se o pareamento é possível
paired_ok = all(t in pivot.columns for t in treatments) and
pivot[treatments].dropna().shape[0] ≥ 2

if paired_ok:
    df = pivot[treatments].dropna()
    stat, pvalue = stats.ttest_rel(df[treatments[0]], df[treatments[1]])
    test_used = "teste t pareado (sobre as médias das replicatas)"
else:
    grp = [ReplicateAverages.loc[ReplicateAverages["Treatment"]==t, "RepMean"]
    for t in treatments]
    stat, pvalue = stats.ttest_ind(grp[0], grp[1], equal_var=False)
    test_used = "teste t de Welch (sobre as médias das replicatas)"

print(test_used, "p =", pvalue)
```

(Opcional) Tamanho de efeito calculado a partir das médias das replicatas:

```
def cohens_d(a, b):
    a, b = np.asarray(a), np.asarray(b)
    na, nb = len(a), len(b)
    sa, sb = np.var(a, ddof=1), np.var(b, ddof=1)
    sp = np.sqrt(((na-1)*sa + (nb-1)*sb) / (na+nb-2))
    return (np.mean(b) - np.mean(a)) / sp
```

```
if paired_ok:
    # Em desenhos pareados, relatar também a média das diferenças
    diff = df[treatments[1]] - df[treatments[0]]
    print("Média das diferenças pareadas =", diff.mean())
else:
    d = cohens_d(grp[0], grp[1])
    print("Cohen's d =", d)
```

0.10 Etapa 8: Criar o SuperPlot

Os pontos pequenos representam células individuais; os pontos grandes e contornados representam as médias das replicatas, coloridas de acordo com a replicata. Opcionalmente, as médias pareadas podem ser conectadas por linhas.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid", context="talk")

fig, ax = plt.subplots(figsize=(7,6))

# 1) Pontos das células individuais com transparência
sns.swarmplot(
    data=combined, x="Treatment", y="Value",
    hue=None, dodge=False, size=3, alpha=0.6, ax=ax
)

# 2) Sobrepor as médias das replicatas como marcadores grandes e contornados
# Manter as médias em primeiro plano e a legenda legível
mean_ax = sns.swarmplot(
    data=ReplicateAverages, x="Treatment", y="RepMean",
    hue="Replicate", dodge=True, size=12, edgecolor="k", linewidth=1.5, ax=ax
)

# Manter apenas a legenda das replicatas
```

```
handles, labels = mean_ax.get_legend_handles_labels()
if handles:
    ax.legend(handles, labels, title="Replicate", frameon=True, loc="best")
else:
    ax.legend_.remove()

# 3) Opcional: conectar médias pareadas entre os tratamentos
if paired_ok:
    # Posições dos tratamentos no eixo x
    xpos = {t:i for i,t in
enumerate(sorted(ReplicateAverages["Treatment"].unique()))}
    for rep, sub in ReplicateAverages.groupby("Replicate"):
        if set(treatments).issubset(set(sub["Treatment"])):
            x1, y1 = xpos[treatments[0]],
float(sub.loc[sub["Treatment"]==treatments[0], "RepMean"])
            x2, y2 = xpos[treatments[1]],
float(sub.loc[sub["Treatment"]==treatments[1], "RepMean"])
            ax.plot([x1, x2], [y1, y2], lw=1.25, alpha=0.85)

# Rótulos e título
ax.set_xlabel("Treatment")
ax.set_ylabel("Value")
ax.set_title("SuperPlot: células individuais + médias das replicatas")

# 4) Anotar o valor de p acima dos dois grupos
ymax = max(combined["Value"].max(), ReplicateAverages["RepMean"].max())
line_y = ymax + 0.05*(ymax if ymax != 0 else 1)
ax.plot([0, 0, 1, 1], [line_y, line_y*1.02, line_y*1.02, line_y], lw=1.5, c="k")
ax.text(0.5, line_y*1.04, f"p = {pvalue:.3g}\n{test_used}", ha="center",
va="bottom")

plt.tight_layout()
plt.show()
```

Consistência das cores entre os tratamentos: usar `hue="Replicate"` garante que cada replicata mantenha a mesma cor em todas as condições.

0.11 Etapa 9: Salvar figuras prontas para publicação

```
plt.savefig("superplot.png", dpi=300, bbox_inches="tight")
plt.savefig("superplot.svg", bbox_inches="tight") # vector
```

0.12 Etapa 10: Transformar o código em uma função reutilizável

Insira esta função em um script ou notebook e chame `make_superplot(...)` para diferentes conjuntos de dados.

```
def make_superplot(
    df, value_col="Value", treatment_col="Treatment", replicate_col="Replicate",
    treatments=None, connect_paired=True, outfile=None, figsize=(7,6)
):
    import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt
    import seaborn as sns
    from scipy import stats

    if treatments is None:
        treatments = list(df[treatment_col].unique())
        if len(treatments) != 2:
            raise ValueError("This helper currently supports exactly two
                               treatments. Provide `treatments=[A,B]`.")

    # Médias das replicatas
    rep_means = (
        df.groupby([treatment_col, replicate_col], as_index=False)[value_col]
        .mean()
        .rename(columns={value_col: "RepMean"})
    )

    # Estatística sobre as médias das replicatas
    pivot = rep_means.pivot_table(index=replicate_col, columns=treatment_col,
                                   values="RepMean")
```

```

paired_ok = all(t in pivot.columns for t in treatments) and
pivot[treatments].dropna().shape[0] ≥ 2
if paired_ok:
    sub = pivot[treatments].dropna()
    stat, pvalue = stats.ttest_rel(sub[treatments[0]], sub[treatments[1]])
    test_used = "teste t pareado (médias das replicatas)"
else:
    g0 = rep_means.loc[rep_means[treatment_col]==treatments[0], "RepMean"]
    g1 = rep_means.loc[rep_means[treatment_col]==treatments[1], "RepMean"]
    stat, pvalue = stats.ttest_ind(g0, g1, equal_var=False)
    test_used = "teste t de Welch (médias das replicatas)"

sns.set(style="whitegrid", context="talk")
fig, ax = plt.subplots(figsize=figsize)

sns.swarmplot(data=df, x=treatment_col, y=value_col, hue=None, size=3,
alpha=0.6, ax=ax)
mean_ax = sns.swarmplot(
    data=rep_means, x=treatment_col, y="RepMean",
    hue=replicate_col, dodge=True, size=12, edgecolor="k", linewidth=1.5,
ax=ax
)

handles, labels = mean_ax.get_legend_handles_labels()
if handles:
    ax.legend(handles, labels, title="Replicate", frameon=True, loc="best")
else:
    ax.legend_.remove()

if connect_paired and paired_ok:
    xpos = {t:i for i,t in
enumerate(sorted(rep_means[treatment_col].unique()))}
    for rep, sub in rep_means.groupby(replicate_col):
        if set(treatments).issubset(set(sub[treatment_col])):
            x1 = xpos[treatments[0]]
            x2 = xpos[treatments[1]]

```

```

        y1 = float(sub.loc[sub[treatment_col]==treatments[0], "RepMean"])
        y2 = float(sub.loc[sub[treatment_col]==treatments[1], "RepMean"])
        ax.plot([x1, x2], [y1, y2], lw=1.25, alpha=0.85)

    ax.set_xlabel("Treatment")
    ax.set_ylabel(value_col)
    ax.set_title("SuperPlot: células individuais + médias das replicatas")

    ymax = max(df[value_col].max(), rep_means["RepMean"].max())
    line_y = ymax + 0.05*(ymax if ymax != 0 else 1)
    ax.plot([0, 0, 1, 1], [line_y, line_y*1.02, line_y*1.02, line_y], lw=1.5,
c="k")
    ax.text(0.5, line_y*1.04, f"p = {pvalue:.3g}\n{test_used}", ha="center",
va="bottom")

plt.tight_layout()
if outfile:
    plt.savefig(outfile, dpi=300, bbox_inches="tight")
return fig, ax, {"pvalue": float(pvalue), "test": test_used,
"replicate_means": rep_means}

```

Uso:

```

fig, ax, stats_info = make_superplot(
    combined,
    value_col="Value",
    treatment_col="Treatment",
    replicate_col="Replicate",
    treatments=["Control", "Drug"],
    connect_paired=True,
    outfile="superplot.png")
print(stats_info)

```

0.13 Etapa 11: Variações comuns

- Nome diferente para a métrica: altere `value_col` para o nome da coluna correspondente, por exemplo, "Speed".
- Escala logarítmica: use `ax.set_yscale("log")` após criar o gráfico.
- Mais de dois tratamentos: continue mostrando todos os dados e as médias das replicatas. Para a análise estatística, realize comparações apropriadas, como Dunnett contra o controle, e aplique correção para testes múltiplos quando necessário, como Benjamini–Hochberg.

0.14 Etapa 12: Recomendações de reprodutibilidade

- Preserve os dados brutos em nível celular e a tabela de médias das replicatas usada na análise estatística.
- Fixe sementes aleatórias quando pertinente. Isso não é necessário para `swarmplot`, mas é uma boa prática em outras análises.
- Exporte tanto a figura, em formatos como `.png` e `.svg`, quanto a tabela `ReplicateAverages` utilizada em Métodos ou nos dados-fonte.

0.14.1 FAQ

- Por que testar as médias das replicatas? Porque as replicatas são a unidade de independência biológica. Testar milhares de células individuais inflaria artificialmente n e subestimaria a incerteza.
- Pareado ou não pareado? Quando a mesma replicata biológica aparece em todas as condições, use um teste pareado. Caso contrário, use o teste de Welch não pareado.